⚡ ExpressKit — Opinionated Backend Starter for Express

TypeScript · Production-Ready · Batteries Included

📌 What Is ExpressKit?

ExpressKit is an **opinionated backend starter package for Express**, designed to do for backend development **what Next.js does for frontend**.

Instead of starting every project with:

- Folder decisions
- Auth setup
- Config hell
- Repeating middleware logic

ExpressKit gives you a **structured, scalable, production-grade backend foundation out of the box**.

> Think: Next.js for Express backends

🚀 Why ExpressKit Exists

Express is powerful — but **too unopinionated**.

Every new backend project repeats:

- Project structure debates
- Middleware wiring
- Environment setup
- Error handling patterns
- Auth + validation boilerplate

ExpressKit solves this by providing conventions, not just utilities.

🎯 Goals of ExpressKit

- ✅ Enforce a clean, scalable backend structure
- ✅ Provide production-ready defaults
- ✅ Stay framework-light (still Express)
- ✅ Be extensible, not restrictive
- ✅ Feel like a *platform*, not a template

🧠 Philosophy (Inspired by Next.js)

Next.js (Frontend)	ExpressKit (Backend)
File-based routing	Convention-based routing
Built-in config	Centralized config system
API routes	Service-based controllers
Middleware pipeline	Global + scoped middleware
Opinionated defaults	Opinionated backend patterns

> ExpressKit doesn't hide Express — it organizes it.

🏗️ High-Level Architecture

Request
↓
Global Middleware
↓
Route Resolver
↓
Controller
↓
Service Layer
↓
Data / External APIs
↓
Standardized Response

Everything follows **one predictable flow**, across every project.

📁 Standardized Project Structure

```
src/
├─ app.ts # App bootstrap
├─ server.ts # Server entry
├─ config/ # Environment & app config
├─ routes/ # Route definitions
├─ controllers/ # Request handlers
├─ services/ # Business logic
├─ middlewares/ # Global & scoped middleware
├─ validators/ # Request validation
├─ utils/ # Helpers
├─ libs/ # DB, cache, external services
├─ types/ # Shared TypeScript types
└─ errors/ # Centralized error handling
```

> This structure scales from MVP → startup → enterprise.

🚦 Routing Convention

Instead of messy route files, ExpressKit enforces:

- Thin controllers
- No business logic in routes
- Clear separation of concerns

```
routes/
└─ user.routes.ts
controllers/
└─ user.controller.ts
services/
└─ user.service.ts
```

🔐 Authentication (Pluggable by Design)

ExpressKit does **not hard-lock you** into one auth provider.

Supported / Planned:

- JWT
- Auth0
- OAuth
- Session-based auth

Why?

Because real products **change auth strategies over time**.

⚙️ Built-In Middleware Stack

Out of the box:

- Request logging
- Centralized error handling
- Async error safety
- Validation middleware
- Auth guards
- Rate limiting (optional)

All middleware follows a **consistent contract**.

🧪 Validation & Type Safety

ExpressKit is **TypeScript-first**.

- Shared request/response types
- Centralized validation layer
- Safer refactors
- Fewer runtime bugs

> Your API contracts become part of your codebase, not documentation.

⚙️ Configuration System

A single source of truth for:

- Environment variables
- App settings
- Feature flags
- Service configs

No more `process.env` scattered everywhere.

🧱 Error Handling (One Way, Everywhere)

- Centralized error classes
- Predictable error responses
- Environment-aware stack traces
- Clean API error format

Every service behaves the same — no surprises.

🌍 Deployment-Ready by Default

ExpressKit is designed to run on:

🔹 Railway / Render

- Backend-first hosting
- Easy environment config
- Auto restarts

🔹 Docker

- Local = production parity
- CI/CD ready
- Kubernetes-friendly

> You don't need to "prepare for deployment" — it already is.

📊 Observability (Planned First-Class Feature)

Designed to integrate with:

- Grafana
- Prometheus
- OpenTelemetry
- Winston / Pino logging

Monitoring is **part of the system**, not an afterthought.

🛠️ Tech Stack

- Express
- TypeScript
- Node.js
- Zod / Joi (validation)
- ESLint + Prettier
- Docker

🧭 Roadmap

Phase 1 — Core Framework ✅

- [x] ~~Standard project structure~~
- [x] ~~Controller–Service pattern~~
- [x] ~~Centralized config~~
- [x] ~~Error handling system~~

Phase 2 — Developer Experience 🚧

- [ ] CLI (`create-expresskit-app`)
- [ ] Code generators
- [ ] Auth presets
- [ ] Logging presets

Phase 3 — Advanced 🚀

- [ ] Async job support
- [ ] Microservice presets

- [ ] API Gateway mode
- [ ] Plugin ecosystem

🧠 Who Is ExpressKit For?

- Backend developers tired of boilerplate
- Teams building scalable APIs
- Startups shipping fast but clean
- Developers who like **structure without magic**

🔥 What Makes ExpressKit Different

- Not a template — a **framework mindset**
- Opinionated but flexible
- Scales with your product
- Backend-first philosophy
- Designed like real production systems

👨‍💻 Author

Prathmesh Desai

AI Systems Engineer · Full-Stack Developer

> ExpressKit aims to become the default starting point for serious Express backends, just like Ne